LRU Cache Implementation

- **Problem:** Build an LRU (Least Recently Used) cache and use it in a React app to cache API responses, with a live view of what is cached.
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional from non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and what must happen. This list is the checklist for building and testing.
3. **Design the architecture.** Name the modules and give each one job.
4. **Design state and data.** Decide what is stored, what is derived and who owns it.
5. **Define contracts.** Function signatures, payload shapes, mock API.
6. **Trace one flow end to end** before writing code, to find gaps.
7. **Find the hard parts early** and decide how you will solve them.
8. **Build in layers.** Smallest working version first, then add one working layer at a time.
9. **Cover states and edge cases.** Empty, loading, error, limits.
10. **Wrap up.** Explain trade-offs and what you would add with more time.

Time budget (60 minutes)

Phase	Minutes
Requirements, questions, assumptions	5
Architecture, state, contracts	7
`LRUCache` class (core data structure)	15
Cached fetcher (dedupe, stats, subscribe)	10
React UI (lookup + cache panel)	15
Edge cases, polish, accessibility	5
Wrap-up and trade-offs	3

Think out loud

Say every decision and the reason as you make it: "I'll use a Map plus a doubly linked list because both get and put must be O(1)". The interviewer grades your reasoning as much as the code. If you get stuck, say what you are trying and what you would check next.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- An `LRUCache` with a fixed **capacity** and `get(key)` / `put(key, value)`.
- `get` returns the value and marks the key as **most recently used** (MRU). Missing key returns `undefined`.
- `put` inserts or updates. When the cache is over capacity, it **evicts the least recently used** (LRU) entry.
- Use it in a React app: look up a user by id. A repeated lookup is served from the cache instantly; a new one goes to the (mock) API.
- Show the cache contents ordered MRU to LRU, the entry that will be evicted next, and hit / miss / eviction counts.
- Change capacity at runtime, remove one entry, clear the cache.

Non-functional requirements

- **Performance:** `get` and `put` in **O(1)** time. O(capacity) memory.
- **Correctness under async:** two lookups of the same id at the same time make one request; a slow old response must not overwrite a newer one.
- **Error handling:** failed requests are shown and never cached. Invalid ids are rejected before any request.
- **Accessibility:** labelled input, results announced with `aria-live`, keyboard usable buttons, error with `role="alert"`.
- **Reusability:** the cache class knows nothing about React or users. The cached fetcher works with any async function.
- **Extensibility:** easy to add TTL, max size in bytes, or persistence later.

Clarifying questions

1. Should `get` on a missing key return `undefined`, `null` or `-1` (LeetCode style)?
2. Does `has(key)` count as a "use" and change recency? (We say no.)
3. Does updating an existing key with `put` make it most recently used? (We say yes.)
4. Should errors be cached (negative caching)? (We say no.)
5. Do entries expire after some time (TTL), or only by capacity?
6. Is the cache shared across the whole app or per component?

Assumptions

- Missing key returns `undefined`; capacity is a positive integer, default 3.
- `get` and `put` both move the key to MRU; `has` does not.
- Only successful responses are cached; no TTL in the first version.
- One app-wide cache (module singleton) for users.
- Mock API with 400-1000 ms latency; ids divisible by 13 fail so we can show the error state.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Fetch id `1` (not cached)	"Loading...", then user shown with **NETWORK** badge; miss +1; `#1` appears at the top (MRU)
Fetch id `1` again	Shown instantly with **CACHE HIT** badge; hit +1; `#1` moves to the top
Fetch ids 1, 2, 3 then 4 with capacity 3	`#1` (LRU) is evicted; eviction +1; list is 4, 3, 2
Fetch 1, 2, 3, then 1 again, then 4	`#2` is evicted (1 was used recently, so it is no longer LRU)
Click `#1` twice quickly (not cached)	Only one network request; both clicks get the same result
Fetch 5 (slow), then quickly fetch 1 (cached)	Shows user 1; when 5 resolves later it is cached but does not replace the result on screen
Fetch id `13`	Error "User 13 not found"; nothing added to cache
Enter `0`, `101`, `abc`, `2.5`	Validation error, no request
Change capacity 3 -> 1	LRU entries evicted until 1 is left; evictions counted
Click ✕ on an entry	Entry removed; next fetch of that id is a miss
Click **Clear**	Cache and stats reset; a response still in flight is not added afterwards
Cache full	Last entry shows a "next to evict" badge

3. Design the architecture

Module	Responsibility
`lib/LRUCache.js`	Pure data structure: O(1) get/put/delete, eviction, MRU -> LRU listing
`lib/createCachedFetcher.js`	Wraps any async fetcher with the LRU cache, in-flight dedupe, stats and a subscribe API
`api/fetchUser.js`	Mock API with latency and a failure case
`store/userCache.js`	The one app-wide cached fetcher for users
`hooks/useCacheSnapshot.js`	Connects the store to React with `useSyncExternalStore`
`components/UserLookup.jsx`	Form, quick buttons, loading / error / result states
`components/CachePanel.jsx`	Cache list, stats, capacity select, remove and clear
`App.jsx`	Page layout

```
App
├─ UserLookup ─── userCache.get(id) ──────┐
└─ CachePanel ─── useCacheSnapshot(userCache)
                   setCapacity / invalidate / clear
```

```
                                  |
        store/userCache.js  (createCachedFetcher(fetchUser, { capacity: 3 }))
             ├── LRUCache          (Map + doubly linked list)
             ├── inflight Map      (key -> Promise)
             └── fetchUser(id)     (mock API)
```

Why this split: the hardest logic (LRU ordering, dedupe, race handling) lives in plain JavaScript that we can test with `node` and no browser. React only reads a snapshot and calls methods. Start with `LRUCache` alone; add the fetcher layer only once the class works.

---

4. Design the state / data model

Inside `LRUCache`

Field	Purpose
`capacity`	Max number of entries
`map: Map<key, Node>`	O(1) lookup of the node for a key
`head`, `tail` (sentinels)	Ends of the doubly linked list; `head.next` is MRU, `tail.prev` is LRU
`Node { key, value, prev, next }`	One entry; keeps `key` so eviction can delete it from the map
`onEvict`	Optional callback, used for the eviction counter

Inside the cached fetcher (store)

Field	Purpose
`cache`	The `LRUCache` instance
`inflight: Map<key, Promise>`	Requests in progress, for dedupe
`stats { hits, misses, evictions }`	Counters shown in the UI
`generation`	Bumped on `clear()`; responses from an older generation are dropped
`listeners: Set`	Subscribers (React) to notify on change
`snapshot`	Immutable object React reads; rebuilt only when something changes

React component state

State	Owner	Purpose
`input`	`UserLookup`	Controlled input value
`status`	`UserLookup`	`idle` / `loading` / `success` / `error`
`result { user, fromCache, ms }`	`UserLookup`	What to show
`error`	`UserLookup`	Error message

State	Owner	Purpose
`latestRequest` (ref)	`UserLookup`	Id of the newest lookup, to ignore stale responses
cache snapshot	store (read via hook)	Entries, size, capacity, stats, pending

```
// The shape React reads
const snapshot = {
  entries: [[3, user3], [2, user2]], // MRU -> LRU
  size: 2,
  capacity: 3,
  stats: { hits: 4, misses: 3, evictions: 1 },
  pending: [5], // keys in flight
};
```

Senior notes

- **Derived, not stored:** hit rate (`hits / (hits + misses)`), "is full" (`size === capacity`), which item is MRU (index 0) and LRU (last index). Storing them would let them go out of sync.
- **The cache is not React state.** It is a mutable object outside React. Putting a class instance in `useState` and mutating it would not re-render. The store owns it; React subscribes.
- **`status` is one field, not `isLoading` + `isError` booleans**, so impossible combinations cannot happen.

---

5. Design the data structures and contracts

Why Map + doubly linked list

Option	get	put / evict	Problem
Array of entries	O(n) find	O(n) move to front	Too slow
Map only, timestamps	O(1)	O(n) find oldest	Eviction scans everything
Map + doubly linked list	**O(1)**	**O(1)**	Slightly more code
JS `Map` insertion order (delete + set)	O(1)	O(1)	Works in JS, but interviewers usually want the explicit structure

The map finds the node in O(1). The doubly linked list moves a node to the front or removes the tail in O(1), because each node knows its `prev` and `next`. Sentinel `head` and `tail` nodes remove all "is the list empty?" null checks.

Example data

```
// fetchUser(7) resolves to:
{ id: 7, name: "Zoya #7", email: "user7@example.com", fetchedAt: "10:42:13 AM" }
```

The user `id` is the cache key. It is stable and unique, so it is also the React `key` in the cache list.

Contracts

```
// LRUCache
new LRUCache(capacity: number, { onEvict?: (key, value) => void })
cache.get(key)            // value | undefined   (moves key to MRU)
cache.put(key, value)     // void                (insert/update, may evict)
cache.has(key)            // boolean             (no recency change)
cache.delete(key)         // boolean
cache.clear()             // void
cache.setCapacity(n)      // void                (evicts while over capacity)
cache.entries()           // [key, value][]      (MRU -> LRU)
cache.size                // number

// Cached fetcher
const store = createCachedFetcher(fetcher: (key) => Promise<value>, { capacity })
store.get(key)            // Promise<{ value, fromCache: boolean }>  (rejects on fetch error)
store.invalidate(key)     // remove one entry
store.clear()             // empty cache, reset stats, drop in-flight results
store.setCapacity(n)
store.subscribe(listener) // returns unsubscribe
store.getSnapshot()       // { entries, size, capacity, stats, pending }

// Mock API
fetchUser(id)             // Promise<User>, rejects for id % 13 === 0
```

---

6. Trace the data flow

Interaction: cache is full (1, 2, 3 with capacity 3, MRU = 3), user fetches id 4.

```
User clicks "Fetch" with input "4"
        ↓
UserLookup.lookup("4"): validate -> id = 4, requestId = ++latestRequest
        ↓  setStatus("loading")
userCache.get(4)
        ↓  cache.has(4)? no.  inflight.has(4)? no.
stats.misses++ ; promise = fetchUser(4) ; inflight.set(4, promise) ; emit()
        ↓  snapshot rebuilt -> CachePanel re-renders, shows "In flight: #4"
... 400-1000 ms ...
fetchUser resolves { id: 4, ... }
        ↓  generation unchanged -> cache.put(4, user)
LRUCache.put: new node at head; size 4 > 3 -> evict tail.prev (#1)
        ↓  onEvict -> stats.evictions++
finally: inflight.delete(4) ; emit()
        ↓
CachePanel: list 4, 3, 2 ; "next to evict" on #2 ; Evictions 1
UserLookup: requestId is still the latest -> setResult({ fromCache: false }) ; status "success"
```

```
        ↓
Screen: "NETWORK · 734 ms" and the card for "Rohan #4"
```

Fetch 4 again: `cache.has(4)` is true, so `stats.hits++`, `cache.get(4)` moves it to the front, `emit()`, and the promise resolves at once with `fromCache: true`.

---

7. Important technical challenges

Challenge 1: O(1) get and put with recency order

Problem: with capacity 3 holding 1, 2, 3, the user reads 1 and then adds 4. We must evict 2, not 1. Finding "the oldest" by scanning is O(n); moving an item to the front of an array is O(n).

Solution: `Map<key, Node>` for lookup and a doubly linked list for order. Every access unlinks the node and re-adds it after `head`. Eviction removes `tail.prev`. All pointer updates are constant time.

```
#unlink(node) {
  node.prev.next = node.next;
  node.next.prev = node.prev;
}
#addToFront(node) {
  node.prev = this.head;
  node.next = this.head.next;
  this.head.next.prev = node;
  this.head.next = node;
}
get(key) {
  const node = this.map.get(key);
  if (!node) return undefined;
  this.#unlink(node);
  this.#addToFront(node);
  return node.value;
}
```

The node stores its `key` so that when we evict `tail.prev` we know which map entry to delete.

Challenge 2: making React re-render when a mutable cache changes

Problem: `const [cache] = useState(new LRUCache(3))` then `cache.put(...)`: the object is mutated in place, React sees the same reference, and the panel never updates. `new LRUCache(3)` would also run on every render.

Solution: keep the cache in a small external store with `subscribe` and `getSnapshot`, and read it with `useSyncExternalStore`. Each change builds a **new** snapshot object; with no change, `getSnapshot` returns the **same** object (otherwise React loops forever).

```
const emit = () => {
  snapshot = makeSnapshot();          // new object only when something changed
  listeners.forEach((listener) => listener());
```

```
};

export function useCacheSnapshot(store) {
  return useSyncExternalStore(store.subscribe, store.getSnapshot);
}
```

Challenge 3: duplicate requests and stale responses

Problem A (duplicates): the user double-clicks `#5`. Both clicks miss the cache, so we fire two identical requests. **Problem B (stale):** user fetches 5 (slow, 1 s), then 1 (cached, instant). When 5 finally resolves, it overwrites the screen with user 5 even though the user asked for 1 last.

Solution A: keep an `inflight` map of key -> promise. A second miss for the same key returns the existing promise. **Solution B:** each lookup gets an increasing `requestId`; only the latest one may update the UI. The response is still cached, it just is not shown.

```
if (inflight.has(key)) return inflight.get(key);

// in UserLookup
const requestId = ++latestRequest.current;
const { value, fromCache } = await userCache.get(id);
if (requestId !== latestRequest.current) return; // a newer lookup won
```

Challenge 4: clear and capacity change while requests are in flight

Problem: user fetches 4 and clicks **Clear** before it returns. When 4 resolves, `cache.put(4)` puts it back, so the "cleared" cache is not empty. Separately, lowering capacity from 5 to 2 must evict 3 entries immediately, not on the next put.

Solution: a `generation` counter bumped by `clear()`; a response only writes to the cache if its generation still matches. `setCapacity` evicts from the LRU end in a loop.

```
const gen = generation;
fetcher(key).then((value) => {
  if (gen === generation) cache.put(key, value);
  return { value, fromCache: false };
});

setCapacity(capacity) {
  assertCapacity(capacity);
  this.capacity = capacity;
  while (this.map.size > capacity) this.#evictLRU();
}
```

8. Implement in the correct order

#	Step	Working result
1	Vite + React + Tailwind setup, `App` with a heading	Page renders
2	`LRUCache` : `Node` , sentinels, `#unlink` , `#addToFront`	List helpers ready
3	`get` , `put` with eviction	Core LRU works; check with the LeetCode 146 sequence in the console
4	`has` , `delete` , `clear` , `setCapacity` , `entries` , `onEvict`	Full class
5	Mock `fetchUser` with delay and failure	Fake API
6	`createCachedFetcher` : hit / miss path and stats	Second fetch of an id is instant
7	`subscribe` / `getSnapshot` + `useCacheSnapshot`	UI can read the cache
8	`UserLookup` : form, validation, status states, result card	Usable lookup
9	`CachePanel` : list MRU -> LRU, stats, badges	Cache visible, evictions visible
10	Capacity select, remove one, clear	Runtime control
11	In-flight dedupe, latest-request guard, generation on clear	Async edge cases safe
12	Accessibility and empty states	Polished

9. Cover all states and edge cases

State / edge case	Handling
Initial	"Pick an id to fetch a user." and "Cache is empty."
Loading	"Loading...", Fetch button disabled, panel shows "In flight: #id"
Success from network	Amber **NETWORK** badge with time in ms
Success from cache	Green **CACHE HIT** badge, ~0 ms
Error from API (id 13)	Red message with `role="alert"` ; not cached; next try fetches again
Invalid input (empty, 0, 101, 2.5, text)	Validation message, no request
Cache empty	"Cache is empty." text
Cache full	"next to evict" badge on the last entry
Capacity 1	Every new id evicts the previous one
Capacity lowered below size	Evict LRU entries until it fits; evictions counted

State / edge case	Handling
`put` existing key	Value updated, moved to MRU, no eviction
Same id clicked twice quickly	One request (in-flight dedupe)
Older response arrives after newer lookup	Cached, but not shown (latest-request guard)
Clear while a request is in flight	Late response dropped (generation check)
Invalid capacity (0, -1, 1.5)	`RangeError` from the class
Value is `undefined` or `null`	`has` is used for hit detection, so falsy values still count as hits

10. Final structure

```
_96_LRU_Cache_Implementation/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx
    ├── api/
    │   └── fetchUser.js          # mock API
    ├── lib/
    │   ├── LRUCache.js           # pure data structure
    │   └── createCachedFetcher.js# cache + dedupe + stats + subscribe
    ├── store/
    │   └── userCache.js          # app-wide instance
    ├── hooks/
    │   └── useCacheSnapshot.js   # useSyncExternalStore bridge
    └── components/
        ├── UserLookup.jsx
        └── CachePanel.jsx
```

11. Full working code

Setup: `npm create vite@latest` (React, JavaScript), then `npm i tailwindcss @tailwindcss/vite`.

vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
```

```
  plugins: [react(), tailwindcss()],
});
```

src/index.css

```
@import "tailwindcss";

body {
  @apply bg-slate-100;
}
```

src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

src/lib/LRUCache.js

```
// Doubly linked list node: one per cached key
class Node {
  constructor(key, value) {
    this.key = key;
    this.value = value;
    this.prev = null;
    this.next = null;
  }
}

function assertCapacity(capacity) {
  if (!Number.isInteger(capacity) || capacity < 1) {
    throw new RangeError("capacity must be a positive integer");
  }
}

// HashMap (key -> node) + doubly linked list (recency order).
// head.next = most recently used, tail.prev = least recently used.
export class LRUCache {
  constructor(capacity, { onEvict } = {}) {
    assertCapacity(capacity);
    this.capacity = capacity;
    this.onEvict = onEvict;
```

```
    this.map = new Map();
    // sentinel nodes: no null checks when adding/removing at the ends
    this.head = new Node(null, null);
    this.tail = new Node(null, null);
    this.head.next = this.tail;
    this.tail.prev = this.head;
  }

  get size() {
    return this.map.size;
  }

  #unlink(node) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
  }

  #addToFront(node) {
    node.prev = this.head;
    node.next = this.head.next;
    this.head.next.prev = node;
    this.head.next = node;
  }

  #evictLRU() {
    const lru = this.tail.prev;
    this.#unlink(lru);
    this.map.delete(lru.key);
    this.onEvict?.(lru.key, lru.value);
  }

  // O(1). Reading a key makes it the most recently used.
  get(key) {
    const node = this.map.get(key);
    if (!node) return undefined;
    this.#unlink(node);
    this.#addToFront(node);
    return node.value;
  }

  // O(1). Does NOT change recency (useful for debugging/UI).
  has(key) {
    return this.map.has(key);
  }

  // O(1). Insert or update; evicts the LRU entry when over capacity.
  put(key, value) {
    const existing = this.map.get(key);
    if (existing) {
      existing.value = value;
      this.#unlink(existing);
      this.#addToFront(existing);
      return;
```

```
    }
    const node = new Node(key, value);
    this.map.set(key, node);
    this.#addToFront(node);
    if (this.map.size > this.capacity) this.#evictLRU();
  }

  delete(key) {
    const node = this.map.get(key);
    if (!node) return false;
    this.#unlink(node);
    this.map.delete(key);
    return true;
  }

  clear() {
    this.map.clear();
    this.head.next = this.tail;
    this.tail.prev = this.head;
  }

  // Shrinking evicts from the LRU end until we fit.
  setCapacity(capacity) {
    assertCapacity(capacity);
    this.capacity = capacity;
    while (this.map.size > capacity) this.#evictLRU();
  }

  // O(n). Ordered MRU -> LRU, for rendering.
  entries() {
    const out = [];
    for (let n = this.head.next; n !== this.tail; n = n.next) {
      out.push([n.key, n.value]);
    }
    return out;
  }
}
```

src/lib/createCachedFetcher.js

```
import { LRUCache } from "./LRUCache.js";

// Wraps any async fetcher with: LRU cache + in-flight dedupe + stats.
// Exposes subscribe/getSnapshot so React can read it via useSyncExternalStore.
export function createCachedFetcher(fetcher, { capacity }) {
  const listeners = new Set();
  const inflight = new Map(); // key -> Promise (dedupes concurrent misses)
  let generation = 0; // bumped on clear(): late responses from before are dropped
  let stats = { hits: 0, misses: 0, evictions: 0 };

  const cache = new LRUCache(capacity, {
    onEvict: () => {
```

```
      stats = { ...stats, evictions: stats.evictions + 1 };
    },
  });

  const makeSnapshot = () => ({
    entries: cache.entries(),
    size: cache.size,
    capacity: cache.capacity,
    stats,
    pending: [...inflight.keys()],
  });
  let snapshot = makeSnapshot();

  // New snapshot object only on change: useSyncExternalStore needs a stable reference
  const emit = () => {
    snapshot = makeSnapshot();
    listeners.forEach((listener) => listener());
  };

  function get(key) {
    if (cache.has(key)) {
      stats = { ...stats, hits: stats.hits + 1 };
      const value = cache.get(key); // moves key to MRU
      emit();
      return Promise.resolve({ value, fromCache: true });
    }
    if (inflight.has(key)) return inflight.get(key);

    stats = { ...stats, misses: stats.misses + 1 };
    const gen = generation;
    const promise = fetcher(key)
      .then((value) => {
        if (gen === generation) cache.put(key, value); // errors are never cached
        return { value, fromCache: false };
      })
      .finally(() => {
        inflight.delete(key);
        emit();
      });
    inflight.set(key, promise);
    emit();
    return promise;
  }

  return {
    get,
    invalidate(key) {
      if (cache.delete(key)) emit();
    },
    clear() {
      generation++;
      cache.clear();
      inflight.clear();
```

```
      stats = { hits: 0, misses: 0, evictions: 0 };
      emit();
    },
    setCapacity(n) {
      cache.setCapacity(n);
      emit();
    },
    subscribe(listener) {
      listeners.add(listener);
      return () => listeners.delete(listener);
    },
    getSnapshot: () => snapshot,
  };
}
```

src/api/fetchUser.js

```
const NAMES = ["Aarav", "Diya", "Kabir", "Meera", "Rohan", "Sara", "Vihaan", "Zoya"];

// Mock API: 400-1000 ms latency, ids divisible by 13 fail (to show the error state)
export function fetchUser(id) {
  return new Promise((resolve, reject) => {
    const delay = 400 + Math.random() * 600;
    setTimeout(() => {
      if (id % 13 === 0) {
        reject(new Error(`User ${id} not found`));
        return;
      }
      resolve({
        id,
        name: `${NAMES[id % NAMES.length]} #${id}`,
        email: `user${id}@example.com`,
        fetchedAt: new Date().toLocaleTimeString(),
      });
    }, delay);
  });
}
```

src/store/userCache.js

```
import { createCachedFetcher } from "../lib/createCachedFetcher.js";
import { fetchUser } from "../api/fetchUser.js";

// One cache for the whole app (module singleton): survives re-renders and is shared
export const userCache = createCachedFetcher(fetchUser, { capacity: 3 });
```

src/hooks/useCacheSnapshot.js

```
import { useSyncExternalStore } from "react";

// Re-renders the component whenever the cache store emits a change
export function useCacheSnapshot(store) {
  return useSyncExternalStore(store.subscribe, store.getSnapshot);
}
```

src/components/UserLookup.jsx

```
import { useRef, useState } from "react";
import { userCache } from "../store/userCache.js";

const QUICK_IDS = [1, 2, 3, 4, 5, 13];

export default function UserLookup() {
  const [input, setInput] = useState("");
  const [status, setStatus] = useState("idle"); // idle | loading | success | error
  const [result, setResult] = useState(null); // { user, fromCache, ms }
  const [error, setError] = useState("");
  const latestRequest = useRef(0);

  async function lookup(rawId) {
    const id = Number(rawId);
    if (!Number.isInteger(id) || id < 1 || id > 100) {
      setStatus("error");
      setError("Enter a whole number from 1 to 100");
      return;
    }

    const requestId = ++latestRequest.current;
    const start = performance.now();
    setStatus("loading");
    setError("");

    try {
      const { value, fromCache } = await userCache.get(id);
      if (requestId !== latestRequest.current) return; // a newer lookup won
      setResult({ user: value, fromCache, ms: Math.round(performance.now() - start) });
      setStatus("success");
    } catch (err) {
      if (requestId !== latestRequest.current) return;
      setError(err.message);
      setStatus("error");
    }
  }

  function handleSubmit(e) {
    e.preventDefault();
    lookup(input);
```

```
  }

  return (
    <section className="rounded-xl border border-slate-200 bg-white p-5 shadow-sm">
      <h2 className="mb-3 text-lg font-semibold">Look up a user</h2>

      <form onSubmit={handleSubmit} className="flex gap-2">
        <label htmlFor="user-id" className="sr-only">User id</label>
        <input
          id="user-id"
          type="number"
          min="1"
          max="100"
          value={input}
          onChange={(e) => setInput(e.target.value)}
          placeholder="User id (1-100)"
          className="flex-1 rounded-lg border border-slate-300 px-3 py-2 focus:outline-none focus:ring-2
focus:ring-indigo-500"
        />
        <button
          type="submit"
          disabled={status === "loading"}
          className="rounded-lg bg-indigo-600 px-4 py-2 font-medium text-white hover:bg-indigo-700
disabled:opacity-50"
        >
          Fetch
        </button>
      </form>

      <div className="mt-3 flex flex-wrap gap-2">
        {QUICK_IDS.map((id) => (
          <button
            key={id}
            type="button"
            onClick={() => {
              setInput(String(id));
              lookup(id);
            }}
            className="rounded-full border border-slate-300 px-3 py-1 text-sm hover:bg-slate-100"
          >
            #{id}
          </button>
        ))}
      </div>

      <div aria-live="polite" className="mt-4 min-h-24">
        {status === "idle" && <p className="text-slate-500">Pick an id to fetch a user.</p>}
        {status === "loading" && <p className="text-slate-500">Loading...</p>}
        {status === "error" && <p role="alert" className="text-red-600">{error}</p>}
        {status === "success" && result && (
          <div className="rounded-lg bg-slate-50 p-3">
            <span
              className={`mb-2 inline-block rounded px-2 py-0.5 text-xs font-semibold ${
```

```
                result.fromCache ? "bg-green-100 text-green-700" : "bg-amber-100 text-amber-700"
              }`}
            >
              {result.fromCache ? "CACHE HIT" : "NETWORK"} · {result.ms} ms
            </span>
            <p className="font-medium">{result.user.name}</p>
            <p className="text-sm text-slate-600">{result.user.email}</p>
            <p className="text-xs text-slate-400">Fetched at {result.user.fetchedAt}</p>
          </div>
        )}
      </div>
    </section>
  );
}
```

src/components/CachePanel.jsx

```
import { userCache } from "../store/userCache.js";
import { useCacheSnapshot } from "../hooks/useCacheSnapshot.js";

const CAPACITIES = [1, 2, 3, 4, 5, 6];

export default function CachePanel() {
  const { entries, size, capacity, stats, pending } = useCacheSnapshot(userCache);
  const total = stats.hits + stats.misses;
  const hitRate = total === 0 ? 0 : Math.round((stats.hits / total) * 100);

  return (
    <section className="rounded-xl border border-slate-200 bg-white p-5 shadow-sm">
      <div className="mb-3 flex items-center justify-between">
        <h2 className="text-lg font-semibold">
          Cache <span className="text-slate-500">({size}/{capacity})</span>
        </h2>
        <div className="flex items-center gap-2">
          <label htmlFor="capacity" className="text-sm text-slate-600">Capacity</label>
          <select
            id="capacity"
            value={capacity}
            onChange={(e) => userCache.setCapacity(Number(e.target.value))}
            className="rounded border border-slate-300 px-2 py-1"
          >
            {CAPACITIES.map((c) => (
              <option key={c} value={c}>{c}</option>
            ))}
          </select>
          <button
            type="button"
            onClick={userCache.clear}
            className="rounded border border-red-300 px-2 py-1 text-sm text-red-600 hover:bg-red-50"
          >
            Clear
          </button>
```

```
        </div>
      </div>

      <dl className="mb-4 grid grid-cols-4 gap-2 text-center text-sm">
        {[
          ["Hits", stats.hits],
          ["Misses", stats.misses],
          ["Evictions", stats.evictions],
          ["Hit rate", `${hitRate}%`],
        ].map(([label, value]) => (
          <div key={label} className="rounded bg-slate-50 p-2">
            <dt className="text-slate-500">{label}</dt>
            <dd className="font-semibold">{value}</dd>
          </div>
        ))}
      </dl>

      {pending.length > 0 && (
        <p className="mb-2 text-sm text-amber-600">In flight: {pending.map((k) => `#${k}`).join(", ")}
</p>
      )}

      {entries.length === 0 ? (
        <p className="text-slate-500">Cache is empty.</p>
      ) : (
        <ol className="space-y-2" aria-label="Cached users, most recently used first">
          {entries.map(([key, user], index) => (
            <li key={key} className="flex items-center justify-between rounded-lg border border-slate-
200 px-3 py-2">
              <span>
                <span className="font-mono text-slate-500">#{key}</span> {user.name}
              </span>
              <span className="flex items-center gap-2">
                {index === 0 && <span className="rounded bg-green-100 px-2 text-xs text-green-
700">MRU</span>}
                {index === entries.length - 1 && size === capacity && (
                  <span className="rounded bg-red-100 px-2 text-xs text-red-700">next to evict</span>
                )}
                <button
                  type="button"
                  aria-label={`Remove user ${key} from cache`}
                  onClick={() => userCache.invalidate(key)}
                  className="text-slate-400 hover:text-red-600"
                >
                  ×
                </button>
              </span>
            </li>
          ))}
        </ol>
      )}
    </section>
```

```
  );
}
```

src/App.jsx

```
import UserLookup from "./components/UserLookup.jsx";
import CachePanel from "./components/CachePanel.jsx";

export default function App() {
  return (
    <main className="mx-auto min-h-screen max-w-4xl p-6 text-slate-800">
      <h1 className="mb-1 text-2xl font-bold">LRU Cache</h1>
      <p className="mb-6 text-slate-600">
        Fetch users by id. Repeat lookups come from the cache; when it is full, the least recently used
user is evicted.
      </p>
      <div className="grid gap-6 md:grid-cols-2">
        <UserLookup />
        <CachePanel />
      </div>
    </main>
  );
}
```

Quick self-check for the pure logic (run with `node`)

```
import assert from "node:assert/strict";
import { LRUCache } from "./src/lib/LRUCache.js";
import { createCachedFetcher } from "./src/lib/createCachedFetcher.js";

// LRUCache basics (LeetCode 146 sequence)
const evicted = [];
const c = new LRUCache(2, { onEvict: (k) => evicted.push(k) });
c.put(1, 1);
c.put(2, 2);
assert.equal(c.get(1), 1);
c.put(3, 3); // evicts 2
assert.equal(c.get(2), undefined);
c.put(4, 4); // evicts 1
assert.equal(c.get(1), undefined);
assert.equal(c.get(3), 3);
assert.equal(c.get(4), 4);
assert.deepEqual(evicted, [2, 1]);
assert.deepEqual(c.entries().map(([k]) => k), [4, 3]);

// update moves to front, no eviction
c.put(3, 30);
assert.deepEqual(c.entries(), [[3, 30], [4, 4]]);
assert.equal(c.size, 2);
```

```
// has does not change recency
c.has(4);
assert.deepEqual(c.entries().map(([k]) => k), [3, 4]);

// delete, clear, shrink capacity
assert.equal(c.delete(4), true);
assert.equal(c.delete(4), false);
c.clear();
assert.equal(c.size, 0);
assert.deepEqual(c.entries(), []);
const s = new LRUCache(4);
[1, 2, 3, 4].forEach((k) => s.put(k, k));
s.setCapacity(2);
assert.deepEqual(s.entries().map(([k]) => k), [4, 3]);
assert.throws(() => new LRUCache(0), RangeError);
assert.throws(() => s.setCapacity(1.5), RangeError);

// Cached fetcher: hits, misses, dedupe, errors not cached, clear drops late responses
let calls = 0;
const wait = (ms) => new Promise((r) => setTimeout(r, ms));
const fetcher = async (id) => {
  calls++;
  await wait(10);
  if (id === 13) throw new Error("not found");
  return { id };
};
const store = createCachedFetcher(fetcher, { capacity: 2 });
let notified = 0;
const unsub = store.subscribe(() => notified++);

const [a, b] = await Promise.all([store.get(1), store.get(1)]);
assert.equal(calls, 1, "concurrent misses share one request");
assert.equal(a, b);
assert.equal(a.fromCache, false);
assert.deepEqual(await store.get(1), { value: { id: 1 }, fromCache: true });
assert.equal(store.getSnapshot().stats.hits, 1);
assert.equal(store.getSnapshot().stats.misses, 1);

await assert.rejects(store.get(13));
assert.equal(store.getSnapshot().size, 1, "errors are not cached");

await store.get(2);
await store.get(3); // evicts 1
assert.deepEqual(store.getSnapshot().entries.map(([k]) => k), [3, 2]);
assert.equal(store.getSnapshot().stats.evictions, 1);

const snap = store.getSnapshot();
assert.equal(store.getSnapshot(), snap, "snapshot is stable without changes");

const late = store.get(4);
store.clear();
await late;
assert.equal(store.getSnapshot().size, 0, "response from before clear() is dropped");
```

```
store.setCapacity(1);
assert.equal(store.getSnapshot().capacity, 1);
assert.ok(notified > 0);
unsub();

console.log("All LRU checks passed");
```

---

12. Wrap-up

Edge cases handled

- O(1) `get` / `put` / `delete`, eviction of the true LRU entry, update of an existing key moves it to MRU.
- Capacity validation and runtime shrinking with eviction.
- Duplicate in-flight requests deduped; stale responses never overwrite a newer lookup.
- Errors not cached; clear during an in-flight request does not resurrect data.
- Falsy cached values still count as hits (`has` instead of truthiness check).
- Empty, loading, error, full states all visible; accessible form and live result region.

Trade-offs to mention

- **Explicit linked list vs JS `Map` order:** a `Map` keeps insertion order, so `delete` + `set` gives LRU in ~15 lines. The linked list is more code but shows you understand the structure and works in any language. Mention both, build the one the interviewer wants.
- **Module singleton store:** simple and shared, but global. For per-feature or per-user caches, create the store in a Context provider instead.
- **Snapshot copies `entries()` on every change:** O(capacity) per change, fine for small capacities used in a UI. For a huge cache, expose only stats to React, or a version number.
- **Errors not cached:** a failing id is retried every time. Negative caching with a short TTL would protect the server.
- **No TTL:** data can be stale forever while it is in the cache.

With more time

- TTL per entry (store `expiresAt`, treat expired as a miss).
- Size-based limit (bytes) instead of entry count.
- Persist to `sessionStorage` / IndexedDB and rehydrate.
- `AbortController` to cancel requests nobody waits for.
- Stale-while-revalidate: show the cached value and refresh in the background.
- Unit tests with Vitest for `LRUCache` and the fetcher, React Testing Library for the UI.
- A generic `useCachedQuery(key, fetcher)` hook on top of the store.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why not just use a plain object or `Map` with no eviction?

Memory grows without limit. In a long session (infinite scroll, search-as-you-type) the cache keeps every response ever fetched. A capacity with LRU eviction keeps memory bounded while keeping the items the user is most likely to need again.

Q2. Why LRU and not LFU or FIFO?

FIFO evicts the oldest inserted item even if it is used constantly. LFU keeps frequently used items but holds on to items that were popular long ago and needs frequency buckets to stay O(1). LRU is simple, O(1), and matches typical UI usage where recent items are likely to be used again.

Q3. What should `get` return for a missing key?

We return `undefined` (like `Map.get`). LeetCode returns `-1` because values are numbers. If `undefined` can be a valid cached value, callers should use `has(key)` first, which is exactly what the fetcher does.

Data structure

Q4. Why a doubly linked list and not a singly linked list?

To remove a node in O(1) we need its previous node. In a singly linked list we would have to walk from the head to find it, which is O(n).

Q5. Why do nodes store the key as well as the value?

Eviction starts from the list (`tail.prev`), not from the map. Without the key on the node, we could not delete the matching map entry in O(1).

Q6. What are the sentinel `head` and `tail` nodes for?

They are dummy nodes that are always there, so the list is never "empty" and there are no special cases for adding the first node or removing the last one. Fewer `if (node === null)` checks means fewer bugs.

Q7. Can you do it with only a JS `Map`?

Yes, because `Map` iterates in insertion order:

```
get(k) {
  if (!this.m.has(k)) return undefined;
  const v = this.m.get(k);
  this.m.delete(k);   // re-insert = move to the end (MRU)
  this.m.set(k, v);
  return v;
}
put(k, v) {
  this.m.delete(k);
  this.m.set(k, v);
```

```
  if (this.m.size > this.cap) this.m.delete(this.m.keys().next().value); // first key = LRU
}
```

Both are O(1) on average. Mention it, but expect the interviewer to ask for the linked list version.

Q8. What is the time and space complexity?

`get`, `put`, `delete`, `has`: O(1). `entries()` and shrinking capacity: O(n). Space: O(capacity) nodes plus the map.

React integration

Q9. Why not keep the cache in `useState`?

State must be immutable for React to detect changes. The cache is mutated in place, so `setState(cache)` with the same reference does nothing. Copying the whole cache on every `get` would also destroy the O(1) benefit.

Q10. Why `useSyncExternalStore` instead of `useState` + `useEffect` subscription?

It is React's built-in way to read external mutable data. It avoids "tearing" in concurrent rendering (two components seeing different versions) and handles subscribe / unsubscribe correctly in StrictMode.

Q11. What happens if `getSnapshot` returns a new object every time?

React compares snapshots with `Object.is`. A new object on every call looks like a change on every render, so React re-renders forever and warns "The result of getSnapshot should be cached". That is why we rebuild the snapshot only inside `emit()`.

Q12. Why a module singleton instead of `useRef` inside a component?

A ref cache dies when the component unmounts and is not shared with other components. The singleton survives navigation and lets both panels read the same data. For multiple independent caches, put the store in a Context.

Async challenges

Q13. Two components request the same id at the same time. How many network calls?

One. The first miss stores its promise in `inflight`; the second finds it and returns the same promise. The entry is removed in `finally`, so a failure does not block retries.

Q14. What if the API is slow and the user fetches several ids quickly?

Each lookup gets a `requestId`; only the latest one updates the UI. Older responses are still cached (useful later) but not shown. With more time, use `AbortController` to cancel requests nobody is waiting for.

Q15. Why not cache errors?

An error is often temporary (network blip). Caching it would keep showing the error until eviction. If the server is overloaded, we could add negative caching with a short TTL.

"What if" twists

Q16. What if data must expire after 5 minutes?

Store `expiresAt` in each node. In `get`, if `Date.now() > expiresAt`, delete the node and treat it as a miss. Optionally run a lazy sweep from the LRU end on `put`.

Q17. What if the limit is memory size, not item count?

Keep a `currentSize` and a `sizeOf(value)` function (for example `JSON.stringify(value).length`). On `put`, add the size and evict from the LRU end while `currentSize > maxSize`. Reject single items larger than the max.

Q18. What if the cache holds 1 million items?

`get` / `put` are still O(1). The UI part is the problem: `entries()` on every change is O(n). Expose only stats and a version to React, and render a virtualised or paginated list. Also consider `WeakRef` or IndexedDB if values are large.

Q19. What if the cache must survive a page reload?

Write `entries()` to `sessionStorage` (debounced) and rebuild by inserting from LRU to MRU on load, so the order is preserved. For large data, use IndexedDB.

Q20. How would you make it generic for any API?

`createCachedFetcher` already takes any `fetcher(key)`. Wrap it in a hook: `useCachedQuery(key, store)` that returns `{ data, status, error }`. For keys made of several params, build a stable string key like `JSON.stringify([url, params])`.

Trade-offs

Q21. Why is `has` not counted as a use?

`has` is a question about the cache, not a read of the value. If it changed recency, debugging tools or UI checks would change eviction order. The fetcher calls `has` and then `get`, so a real read still moves the key.

Q22. How would you test this?

Unit-test `LRUCache` with the LeetCode 146 sequence, update-moves-to-front, delete, clear and shrinking capacity. Test the fetcher with a fake async function: dedupe (call count), errors not cached, clear during in-flight. Test the UI with React Testing Library: second lookup shows "CACHE HIT".